

一、背景与痛点

对于求职网站的深度使用者来说，最常见的困境，从来不是“没有岗位可投”，而是在海量岗位中迷失了时间与精力。

有趣的是，这种困境并非一开始就存在——恰恰相反，它是随着使用深度的增加而逐步加剧的。

1. 初始状态：最理想的“黄金期”

初次使用 LinkedIn 时，你几乎不会感到任何阻力。只需输入 `job_title + Ireland + Posted 24 hours`，搜索结果中每一页的每一个岗位，几乎都值得申请。那种“打开搜索即几乎都是适合的岗位”的流畅感，是所有求职者都渴望的理想状态。

2. 深度使用后：三大困境逐步显现

然而，随着使用时间的拉长，以下三类问题会不可避免地浮现：

（1）困境一：巨头们的广告，有效岗位占比骤降

连续翻过好几页，映入眼帘的却大多是巨头公司的长期岗位广告，当然我觉得这是正常的合理的，对于公司和平台都是双赢的，只是平台没有想过为申请者做一定的过滤，因为三赢才是王炸。这些岗位并非不真实，而是在“今天”这个时间节点上，真正最适合你申请的可能只有其中的 1-2 个。其结果就是：你可能翻阅了五六页，真正完成的投递却寥寥无几。

（2）困境二：重复申请，消耗的不仅是时间

一个更为隐蔽的陷阱是：你明明几天前刚申请过某个岗位，今天它（或高度相似的岗位）又以“Posted in 24 hours”的姿态重新出现在搜索结果中。你觉得“真的很适合”，于是再次提交申请，却在不知不觉中造成了同一家公司的重复投递——这既不体面，也浪费了本应投给新机会的窗口。

而如果你克制住不申请，则只能继续往下翻、继续找。等到回过神来，几个小时已经过去，投递数量却停留在个位数。

（3）困境三：跨平台放大困境，而非解决困境

当你意识到单一平台的局限，转而尝试增加更多平台如 Indeed 和 IrishJobs 来扩大申请广度的时候，等待你的并不是“新大陆”，而是完全相同困境在不同平台上的重复上演。

3. 本工具的起点

这款工具不是为了帮你发现更多岗位，而是帮你更快地判断——在今天的此刻，针对你个人，哪些岗位值得投。

它的核心目标，是让每一位深度求职者重新回到“初始状态”的体验：

一打开结果表，看到的就是你今天最应该申请的那批岗位。点击 url、投递、完成——像第一天使用平台时那样高效、流畅、不纠结。

二、核心理念：规则推荐 ≠ 低端推荐

在推荐系统领域，有一个长期存在的迷思：仿佛只有引入 ML/DL 才算“真正的推荐”。但我的实践经验给出了一个更务实的判断：在大多数需要推荐系统的场景中，基于用户行为和购买心理理解的统计规则，往往可以覆盖 80% 的有效推荐。ML/DL 模型所做的工作，更多是在剩余 20% 的空间中争取边际提升。

而这一边际提升的 ROI，完全取决于人力成本与模型在那 20% 空间中带来的增量收益之间的权衡。

我们这款工具的核心特色，并不在于它是一款“推荐工具”——因为这里的推荐逻辑本质上是建立在个体需求之上的规则推荐。它真正的价值在于：如何在已有招聘平台、工具和资源的基础上，添加一款简洁、可扩展、且能显著提升求职效率的设计方案。

下面我将围绕这一理念，系统介绍本工具的整体架构、方案演进与使用方式。

三、整体架构概览

本工具的核心数据流转可分为以下五个阶段：

- 1. **数据采集**（从招聘平台获取最新岗位的增量数据）
- 2. **历史数据加载**（提取个人过往申请记录，在这里我们使用了 jellyfish.jaro_winkler_similarity 去匹配自己申请数据与实际平台数据中的公司名称,比如我们自己手写是 abc company, 但是平台上是 abc company ireland)
- 3. **数据预处理与特征提取**（平台数据和个人历史数据的清洗、解析、结构化等数据预处理）
- 4. **规则过滤与推荐生成**（基于个人申请行为的定制化筛选，可拓展可修改）
- 5. **投递追踪与总表维护**（记录已投递状态，追加到历史总表）

其设计目标是：将过去 1 个小时只能申请 1-2 个岗位，变成 1 个小时精准投递 5-7 个岗位。

四、方案演进：从“全自动”到“半自动”的务实折中

4.1 初始设计：完全自动化（V1）

我们最初的设计目标，是实现一套完全自动化、无需人工介入的处理流程，具体包括：

- 1. 自动爬取 LinkedIn/ IrishJobs 官网上按发布时间排序的相关岗位增量数据 (按时间倒序获取最新发布的职位)；
- 2. 提取历史申请数据（首次使用时需要做数据预处理，后续直接调用已经处理并保存好的结果表即可）；
- 3. 将昨日实际投递的增量数据追加到总表中（因为需要更新不同公司 last apply date 这一核心特征）；
- 4. 对当日爬取的增量数据进行数据预处理及关键特征提取；
- 5. 基于推荐规则筛选并生成当日的最终推荐增量表；
- 6. 输出 today_recommendation.csv，用户打开即可一键直达申请页面。

这一方案在理想情况下运行流畅，但很快我们遭遇了现实的阻力。

4.2 现实瓶颈：反爬机制的硬限制

在实际运行中，自动化爬取方案面临两大平台的严苛限制：

平台	限制规则
LinkedIn	每页最多返回 10 个岗位，且同样也最多支持 4 页
IrishJobs	最多仅可爬取 4 页，尝试第 5 页会被直接禁止访问，多次访问会封禁 IP

当当日新增岗位数量较多时，爬虫会出现漏采，影响推荐覆盖率的完整性，也直接动摇了工具的通用性与可扩展性。

4.3 方案调整：手动 HTML + 自动解析（V2）

基于上述限制，我们做出了一个务实的折中决策：

放弃全自动爬取，改为“手动下载 HTML 页面 + 自动解析与推荐”的半自动模式。即：

- 1. **手动下载**：筛选相关岗位相关地点并按发布时间排序后，直接另存为 HTML 文件（文件名格式：page_1.html 依此类推），保存至对应的 xxx_raw_data 目录（如 LinkedIn 数据保存至 linkedin_raw_data）；
- 2. **自动解析**：调用对应平台的解析脚本，对 HTML 进行预处理与信息提取；
- 3. **跨平台合并**：合并不同平台的增量数据表，并内置不同平台相同公司岗位的去重逻辑。

这一调整看似“退步”，实则提升了工具的通用性、稳定性和可扩展性：

- 不受平台反爬策略变化的影响；
- 可覆盖任意求职平台（不限 LinkedIn/ IrishJobs）；
- 不限任何适合岗位的关键词使用（你可以先搜索 Agentic 关键词，然后搜索 Machine Learning，然后搜索 Data Scientist，然后搜索 Data Engineering 等，通用性更强）
- 用户无需维护复杂的爬虫脚本；
- 每次运行仅需多花几分钟下载 HTML，而推荐的收益完全覆盖这一成本。
- 任何时间任何地点，当你想要投递的时候，你就可以操作了，非常方便。

这一调整带来的最大收益是：你可以将任何求职平台接入本工具，将公司+岗位作为 unique key 不会出现任何重复的定制化推荐，真正实现跨平台高效求职。

五、推荐规则详解

目前系统内置了三条核心推荐规则，全部基于时间维度与历史申请行为构建（你可以根据自己的投递规则进行修改

与拓展):

规则编号	条件	推荐度	设计逻辑
1	近一周内发布 + 从未申请过该公司	必申请	新公司意味着新机会，重点是向对方公司告知你在看新机会
2	近 3 天内发布 + 距离最后一次申请已超 30 天	必申请	避免短期内重复投递同一家公司，给双方合理的间隔期
3	所有推荐岗位按 posted time 降序排列	优先级	最新发布岗位回复率通常更高

关键设计细节:

- 今日推荐表中，所有岗位的申请日期均标记为今天（因为假设你今天全部申请掉）；
- 在你实际申请中，如果未投递的岗位则可以直接移除预置日期，后续追加到总表中的时候，我们会将实际投递的追加到总表中，这样你今天没有投递的公司后续仍可重新进入推荐列表；
- 对于你通过其他途径申请的岗位，比如你通过 job alerts 申请的时候，你可以手动追加到结果表中即可。

六、效率对比

指标	纯平台申请	结合使用本工具
1 小时可完成投递数	1-2 个	6 个左右
跨平台覆盖能力	单平台为主，跨平台岗位重复多	多平台自动整合
推荐精准度	深度使用需依赖人工判断	规则一致、无遗漏

以我个人实际使用为例:

- 原先手动申请，1 小时；
- 使用本工具后，1 小时可完成 5-6 个岗位的申请。

相当于将有效投递效率提升了好几倍，且额外节省出的时间可用于岗位调研、面试准备、自己感兴趣的项目开发与研究或休息恢复等。

七、使用指南

7.1 适用人群

本工具最适合以下场景:

- 你在 LinkedIn / IrishJobs 上搜索岗位后，1 小时内仅完成 1-2 个投递；
- 你希望系统化地管理自己的申请记录，避免重复投递；
- 你同时使用多个招聘平台，希望统一管理推荐结果。

⚠️ 如果你已能达到 1 小时投递 5-7 个岗位的效率，说明当前方法已接近高效上限，本工具的边际收益可能有限。

7.2 环境要求

依赖	版本
Python	3.12.7
Poetry	2.4.1

7.3 安装与运行

1. 下载代码并解压（未安装 Git 的用户可直接下载 ZIP）

2. 进入项目目录

```
cd your-project-directory
```

3. 安装依赖

```
poetry install --no-root
```

4. 从 LinkedIn 和 Irishjobs 官网上根据要求搜索岗位，然后另存为 html 文件（page_1.html 依次命名）到对应的 xxx_raw_data 文件夹中。

5. 打开根目录下的 one_click_run.sh，修改里面的 linkedin_start_page, linkedin_end_page, irishjobs_start_page, irishjobs_end_page 参数，分别对应你下载的 html 的页数的起止页数，比如你 linkedin 今天下载了 2 页，那么 linkedin_start_page=1, linkedin_end_page=2。

6. 一键运行

```
poetry run bash one_click_run.sh
```

7.4 获取推荐结果

运行完成后，项目根目录会生成：

today_recommendation.csv

打开此文件，点击 url 列即可直接跳转至对应岗位的申请页面。

八、扩展性与后续计划

8.1 已实现的扩展能力

- 支持多平台接入（你可以新增 Indeed 平台，只要再加一个自动化 parser.py 脚本即可）；
- 内置跨平台岗位去重逻辑，避免同一岗位重复推荐；
- 历史申请数据持久化存储，支持增量更新。

8.2 未来可能的改进方向

- 希望对应的平台可以将这一套功能内置到平台作为一个申请人可以选择的辅助服务功能。

九、写在最后

做这款工具的过程中，最深的体会是：**效率提升的核心，不是做更多的事，而是做更少的决策。**

当机器替你完成了“这个值不值得投”的判断，你就可以把全部精力留给“如何更好地完成这次投递”。

如果你也在求职的路上感到疲惫，不妨试一下这套方法。

十、申明

（1）整套架构设计都是来自个人深度求职的经历，但是其中 parser.py 解析器是来自 Claude Code 协助完成和 one_click_run.sh 是来自 DeepSeek 协助完成的，中文语言优化由 DeepSeek 协助完成，英文翻译是由 ChatGPT 协助完成的。所以这也让我想到了，每一款市面上的 GenAI 都有其独特的优势，所以对于使用者是尽可能地融合使用这些 GenAI 地最大特色，而作为 GenAI 公司应该将这个独特的优势作为他们最强有力地护城河之后再次拓展边界。

（2）目前针对平台的解析器都完全可用，不排除以后平台有变化，届时可以通过 GenAI 帮助优化代码即可。

（3）希望我们每个在寻找工作的人都能够找到满意的工作，努力加油共勉！

GitHub 仓库

代码已开源，欢迎使用、提 Issue 或贡献代码：

 **仓库地址：** https://github.com/QinDSHub/TurboApply_Lightweight_Tool